

TaskFlow Pro — Full Stack Learning Roadmap

Project Overview

Project Name: TaskFlow Pro — Team Productivity SaaS

Description: A full-stack project management app (like a mini Asana/Notion) that covers every major full-stack concept in one coherent, real-world product.

Tech Stack:

- Frontend: React + Redux Toolkit
 - Backend: Node.js + Express
 - Database: MongoDB
 - Cache / Sessions: Redis
 - Payments: Stripe
-

Why This Project?

Every feature maps to a real product need — not a contrived exercise. You are building something a real user would pay for.

- A to-do app is too small (no payments, no auth complexity)
 - An e-commerce app misses time tracking and calendar
 - A social media clone skips Redis and pagination depth
 - TaskFlow hits everything in one coherent product
-

Skill-to-Feature Mapping

Skill	Feature in TaskFlow Pro
Redis	Session store + rate limiting + active timer caching
MongoDB	Tasks, users, time logs with aggregations
Google One Tap / OAuth	Real OAuth flow on the login page
Debouncing	Search bar, auto-save draft tasks

Skill	Feature in TaskFlow Pro
Throttling / Rate Limiting	API protection on login and form submit
Pagination	Tasks list (cursor-based for efficiency)
Calendar	Task due dates, drag-to-reschedule
Time Tracking	Hooks + Redux + Redis TTL
React Suspense	Lazy load routes + data fetching states
Context API	Theme and auth user available globally
Redux + RTK Query	Server state management, optimistic updates
Promises	Parallel data loads with Promise.all
Stripe	Pro plan upgrade, webhook listener on backend
Forms	React Hook Form + Zod validation
Lazy Loading	Code splitting by route with React.lazy
Custom Hooks	useTimer, useDebounce, usePagination
Error Boundaries	Catch crashes without full-page failures
Theme Provider	Dark/light mode with CSS variables + Context

Phase 1 — Auth & User Foundation (Week 1-2)

Goal

Set up the project, connect MongoDB, build the full authentication system.

Features

1. Email + Password Login

- Register, login, logout with hashed passwords using bcrypt
- Issue JWT access token (short-lived) and refresh token (long-lived)
- Store refresh tokens in Redis with TTL
- Technologies: Node.js, Express, MongoDB, Redis, bcrypt, jsonwebtoken

2. Google One Tap + OAuth Login

- Integrate Google OAuth 2.0 using passport.js
- One-tap login on the frontend — user clicks, they're authenticated
- On success, create or find user in MongoDB, issue JWT
- Technologies: passport.js, Google OAuth 2.0, React

3. Rate Limiting & Throttling

- Protect /login and /register endpoints from brute force attacks
- Track attempt counts per IP address in Redis with a rolling window
- Return 429 Too Many Requests after threshold is exceeded
- Technologies: express-rate-limit, Redis, ioredis

Key Concepts Learned

- How JWT access + refresh token rotation works
 - How OAuth 2.0 flow works end to end
 - How Redis is used as a fast, ephemeral store for rate limits and sessions
-

Phase 2 — Tasks, Forms & Real-time UX (Week 3-4)

Goal

Build the core task management features with polished form handling and real-time UI feedback.

Features

1. Task CRUD with Validated Forms

- Create, edit, delete tasks with full form validation
- Use React Hook Form for form state management
- Use Zod for schema validation (errors show inline)
- Wrap shared state (current user, filters) in Context API
- Technologies: React Hook Form, Zod, Context API, MongoDB

2. Search with Debouncing

- Search tasks by title in real-time as the user types
- Debounce keystrokes (300ms delay) to avoid spamming the API
- Backend uses MongoDB text index for efficient full-text search
- Technologies: useEffect, custom useDebounce hook, MongoDB text index

3. Optimistic Updates

- When user marks a task as complete, update the UI instantly
- If the API call fails, roll back the change automatically
- Makes the app feel instant and responsive
- Technologies: Redux Toolkit, RTK Query optimistic update pattern

Key Concepts Learned

- Controlled vs uncontrolled forms and when to use each
 - How debouncing works and how to implement it as a custom hook
 - Optimistic UI pattern — update first, reconcile later
-

Phase 3 — Performance & Data Loading (Week 5-6)

Goal

Make the app fast and scalable with proper data loading patterns and code splitting.

Features

1. Pagination

- Paginate the tasks list with cursor-based pagination (more efficient than offset)
- Frontend shows Load More button or infinite scroll
- React Suspense handles the loading state between pages
- Technologies: MongoDB cursor pagination, React Suspense, RTK Query

2. Lazy Loading & Code Splitting

- Split the app bundle by route using React.lazy and dynamic imports
- Dashboard, Calendar, Payments pages load on demand — not on first load
- Vite handles chunk splitting automatically

- Technologies: React.lazy, Suspense, Vite, dynamic import()

3. Theme Provider & Context

- Dark/light mode toggle stored in ThemeContext
- CSS variables switch the entire app color scheme with one class change
- User preference persisted in localStorage and synced to MongoDB user profile
- Technologies: React Context API, CSS variables, localStorage

Key Concepts Learned

- Cursor-based vs offset pagination and why cursor wins at scale
 - How code splitting reduces initial bundle size and speeds up first load
 - How Context API propagates state without prop drilling
-

Phase 4 — Calendar & Time Tracking (Week 7-8)

Goal

Add the calendar view and time tracking features — the most complex interactive parts of the app.

Features

1. Task Calendar View

- Render tasks on a calendar grid by their due date
- Click a day to see tasks due that day in a side panel
- Drag a task to a new day to reschedule it (updates MongoDB)
- Technologies: react-calendar or react-big-calendar, date-fns, MongoDB

2. Time Tracking

- Start and stop a timer on any task
- Active timers are cached in Redis (fast reads, survives page refresh)
- On stop, write the time log entry to MongoDB
- Show total time logged per task and per day
- Technologies: Custom useTimer hook, Redux, Redis TTL, MongoDB aggregation

3. Custom Reusable Hooks

- Extract all reusable logic into custom hooks
- `useTimer` — start, stop, elapsed time for a task
- `useDebounce` — debounce any value with a configurable delay
- `usePagination` — cursor management and load more logic
- Technologies: React hooks (`useState`, `useEffect`, `useRef`, `useCallback`)

Key Concepts Learned

- How to manage timer state across page navigations using Redux + Redis
 - How to write clean, testable custom hooks
 - MongoDB aggregation pipeline for grouped time totals
-

Phase 5 — Payments, Polish & Ship (Week 9-10)

Goal

Add Stripe payment integration, clean up async flows, add error handling, and ship the app.

Features

1. Stripe Payment Integration

- Free tier (5 tasks max) and Pro tier (unlimited tasks + time tracking)
- Stripe Checkout handles the payment UI — no card data touches your server
- Stripe webhook endpoint on Node.js updates the user tier in MongoDB on successful payment
- Technologies: Stripe.js, Stripe Checkout, Stripe webhooks, Node.js

2. Redux Async Flows with RTK Query

- All API calls go through RTK Query endpoints (not raw fetch/axios)
- Automatic caching, background refetching, and cache invalidation
- Loading, success, and error states are derived automatically — no manual `isLoading` flags
- Technologies: Redux Toolkit, RTK Query, `createApi`, `createAsyncThunk`

3. Promises & Error Boundaries

- On dashboard load, fetch user profile + tasks + calendar events in parallel using Promise.all
- React Error Boundaries catch render crashes in any subtree without taking down the whole app
- Retry logic on failed API calls using RTK Query's retry utilities
- Technologies: Promise.all, React Error Boundary, RTK Query retry

Key Concepts Learned

- How Stripe webhooks work — why you must verify the webhook signature
 - How RTK Query eliminates boilerplate compared to raw useEffect data fetching
 - How Promise.all reduces load time vs sequential await chains
 - How Error Boundaries contain failures gracefully
-

Recommended Learning Order

1. Complete phases in order — each one builds on the last
 2. Each phase adds one new backend concept and one new React concept simultaneously
 3. After Phase 1, you will have a working deployed app (even if minimal)
 4. After Phase 3, the app is fast enough for real users
 5. After Phase 5, the app is monetizable and shippable
-

Suggested Folder Structure

```
taskflow-pro/
├── client/                                (React app)
│   ├── src/
│   │   ├── app/                          (Redux store setup)
│   │   ├── components/                   (Shared UI components)
│   │   ├── features/                     (auth, tasks, calendar, payments)
│   │   ├── hooks/                        (useDebounce, useTimer, usePagination)
│   │   ├── pages/                        (Dashboard, Calendar, Settings, Upgrade)
│   │   ├── context/                      (ThemeContext, AuthContext)
│   │   └── services/                     (RTK Query API definitions)
│   └── server/                           (Node.js + Express)
│       ├── routes/                       (auth, tasks, users, payments)
```

```
|   ├── controllers/      (business logic)
|   ├── models/          (Mongoose schemas)
|   ├── middleware/      (auth guard, rate limiter, error handler)
|   ├── services/        (redis.js, stripe.js, email.js)
|   └── config/          (db.js, passport.js)
|
| └── README.md
```

How to Use This Document with Claude

Upload this file at the start of a new conversation and say something like:

```
| "Here is my learning roadmap for TaskFlow Pro. Let's start with Phase 1. Help me set up the
| Node.js + Express + MongoDB project structure and write the email/password registration
| endpoint."
```

Then work phase by phase. Each session, reference the phase you are on so Claude has full context of where you are in the project.